

TP01 - Relacionamento 1:N

Alunos participantes: Isabel Cristina, Laura Bargas, Pedro Mattar e Yuri Penido.

O EntrePares 1.0 (TP1) é um sistema acadêmico desenvolvido para gerenciar a oferta de cursos livres por alunos da PUC Minas. Ele opera como um banco de dados customizado, realizando a persistência de Usuários e Cursos em arquivos binários de acesso aleatório e otimizando o armazenamento por meio do reaproveitamento de registros excluídos. Para garantir a integridade e a alta performance nas buscas, o sistema emprega criptografia para senhas, Tabelas Hash Extensíveis para buscas diretas (como login e códigos de cursos) e Árvores B+ para ordenação alfabética e para solucionar o relacionamento 1:N entre os criadores e seus respectivos cursos.

Estrutura de Classes Criadas

1. Classes Criadas

- **Principal:** Classe que contém o método main, é responsável por gerenciar o menu controlando o loop principal do programa, as telas de pré-login e pós-login.

```

public class Principal {
    public static void main(String[] args) {
        System.out.println(x: "\nEntrePares 1.0");
        System.out.println(x: "-----");
        System.out.println(x: "\n(A) Login");
        System.out.println(x: "(B) Novo usuario");
        System.out.println(x: "(C) Recuperar senha");
        System.out.println(x: "\n(S) Sair");

        System.out.print(s: "\nOpcao: ");
        String input = console.nextLine();
        if(input.length()==0) continue;
        char ch = input.charAt(index: 0);

        switch(ch) {
            case 'A': case 'a':
                int userId = controleUsuario.login();
                if(userId>0) {
                    System.out.println(x: "Login realizado. Bem-vindo!");
                    // menu após login
                    boolean sairLogado = false;
                    while(!sairLogado) {
                        Console.clear();
                        System.out.println(x: "\nEntrePares 1.0");
                        System.out.println(x: "-----");
                        System.out.println(x: "> Inicio\n");
                        System.out.println(x: "(A) Meus dados");
                        System.out.println(x: "(B) Meus cursos");
                        System.out.println(x: "(C) Minhas inscricoes");
                        System.out.println(x: "\n(S) Sair");
                        System.out.print(s: "\nOpcao: ");
                        String in2 = console.nextLine();

```

- **Console:** Classe utilitária responsável por limpar a tela do terminal.

```

public class Console {
    public static void clear() {
        try {
            String os = System.getProperty(key: "os.name").toLowerCase();
            if (os.contains(s: "win")) {
                // Try to use cmd cls which generally clears the console on Windows
                new ProcessBuilder(...command: "cmd", "/c", "cls").inheritIO().start().waitFor();
            } else {
                // ANSI escape sequence for most UNIX terminals
                System.out.print(s: "\033[H\033[2J");
                System.out.flush();
            }
        } catch (Exception e) {
            // fallback: print many new lines
            for (int i = 0; i < 80; i++)
                System.out.println();
        }
    }
}

```

2. Núcleo de Armazenamento (Pacote aed3):

- **Arquivo<T>**: Implementa o CRUD genérico manipulando arquivos.

```

public class Arquivo<T extends InterfaceEntidade> {
    String nomeEntidade;
    Constructor<T> construtor;
    RandomAccessFile arquivo;
    HashExtensivel<ParIDEndereco> indiceDireto;
    public final int TAM_CABECALHO = 12;

    public Arquivo(String nomeEntidade, Constructor<T> construtor) throws Exception {
        File d = new File(pathname: "dados");
        if (!d.exists())
            d.mkdir();
        d = new File("./dados/"+nomeEntidade);
        if (!d.exists())
            d.mkdir();

        this.nomeEntidade = nomeEntidade;
        this.construtor = construtor;
        arquivo = new RandomAccessFile("./dados/"+nomeEntidade+"/dados.db", mode: "rw");
        if(arquivo.length() < TAM_CABECALHO) {
            arquivo.writeInt(v: 0); // último ID usado
            arquivo.writeLong(-1); // cabeça da lista de espaços vazios
        }
        indiceDireto = new HashExtensivel<> (
            ParIDEndereco.class.getConstructor(),
            n: 4,
            "./dados/"+nomeEntidade+"/indiceDireto.d.db",
            "./dados/"+nomeEntidade+"/indiceDireto.c.db");
    }

    public int create(T entidade) throws Exception {
        arquivo.seek(pos: 0);
    }
}

```

- **HashExtensivel:** Implementação da tabela hash que associa chaves a valores. Cada par de chave e seu valor deve ser armazenado em um endereço específico dessa tabela. Índice indireto, que é identificado por uma chave única. Usuário: e-mail e Curso: código compartilhável.

```
public class HashExtensivel<T extends InterfaceHashExtensivel> {
    public HashExtensivel(Constructor<T> ct, int n, String nd, String nc) throws Exception {
        construtor = ct;
        quantidadeDadosPorCesto = n;
        nomeArquivoDiretorio = nd;
        nomeArquivoCestos = nc;

        arqDiretorio = new RandomAccessFile(nomeArquivoDiretorio, mode: "rw");
        arqCestos = new RandomAccessFile(nomeArquivoCestos, mode: "rw");

        if (arqDiretorio.length() == 0 || arqCestos.length() == 0) {

            diretorio = new Diretorio();
            byte[] bd = diretorio.toByteArray();
            arqDiretorio.write(bd);

            Cesto c = new Cesto(construtor, quantidadeDadosPorCesto);
            bd = c.toByteArray();
            arqCestos.seek(pos: 0);
            arqCestos.write(bd);
        }

        public boolean create(T elem) throws Exception {

            // Carrega TODO o diretório para a memória
            byte[] bd = new byte[(int) arqDiretorio.length()];
            arqDiretorio.seek(pos: 0);
            arqDiretorio.read(bd);
            diretorio = new Diretorio();
        }
    }
}
```

- **ArvoreBMais:** Implementação da ArvoreBMais em que cada nó comporta várias chaves. As árvores B são estruturas planejadas para o acesso de dados em blocos. Usuário: nome e Curso: nome do curso.

```

public class ArvoreBMAis<T> extends InterfaceArvoreBMAis<T> {
    private boolean create1(long pagina) throws Exception {
        // Le a pagina passada como referencia
        arquivo.seek(pagina);
        Pagina pa = new Pagina(construtor, ordem);
        byte[] buffer = new byte[pa.TAMANHO_PAGINA];
        arquivo.read(buffer);
        pa.fromByteArray(buffer);
        int i = 0;
        while (i < pa.elementos.size() && (elemAux.compareTo(pa.elementos.get(i)) > 0)) {
            i++;
        }
        if (i < pa.elementos.size() && pa.filhos.get(index: 0) == -1 && elemAux.compareTo(pa.elementos.get(i)) < 0) {
            cresceu = false;
            return false;
        }
        boolean inserido;
        if (i == pa.elementos.size() || elemAux.compareTo(pa.elementos.get(i)) < 0)
            inserido = create1(pa.filhos.get(i));
        else
            inserido = create1(pa.filhos.get(i + 1));
        if (!cresceu)
            return inserido;

        if (pa.elementos.size() < maxElementos) {
            pa.elementos.add(i, elemAux);
            pa.filhos.add(i + 1, paginaAux);
            arquivo.seek(pagina);
            arquivo.write(pa.toByteArray());
            cresceu = false;
            return true;
        }
    }
}

```

- **ParIDEndereco, ParIdId, ParNomeId:** Classes de suporte para os índices.

```

public class ParIDEndereco implements InterfaceHashExtensivel {

    @Override
    public int hashCode() {
        return this.id;
    }

    public short size() {
        return this.TAMANHO;
    }

    public String toString() {
        return "("+this.id + ";" + this.endereco+")";
    }

    public byte[] toByteArray() throws IOException {
        ByteArrayOutputStream baos = new ByteArrayOutputStream();
        DataOutputStream dos = new DataOutputStream(baos);
        dos.writeInt(this.id);
        dos.writeLong(this.endereco);
        return baos.toByteArray();
    }

    public void fromByteArray(byte[] ba) throws IOException {
        ByteArrayInputStream bais = new ByteArrayInputStream(ba);
        DataInputStream dis = new DataInputStream(bais);
        this.id = dis.readInt();
        this.endereco = dis.readLong();
    }

}

```

2. Entidade Usuário:

- **Usuario:** Entidade com Nome, Email, Hash da Senha e Pergunta Secreta.

```

public class Usuario implements InterfaceEntidade {

    private int idUsuario;
    private String nome;
    private String email;
    private String hashSenha;
    private String perguntaSecreta;
    private String hashRespostaSecreta;

    public Usuario() {
        this(-1, "", email: "", hashSenha: "", pergunta: "", hashResposta: "");
    }

    public Usuario(String nome, String email, String hashSenha, String pergunta, String hashResposta) {
        this(-1, nome, email, hashSenha, pergunta, hashResposta);
    }

    public Usuario(int id, String nome, String email, String hashSenha, String pergunta, String hashResposta) {
        this.idUsuario = id;
        this.nome = nome;
        this.email = email;
        this.hashSenha = hashSenha;
        this.perguntaSecreta = pergunta;
        this.hashRespostaSecreta = hashResposta;
    }

    public int getID() { return idUsuario; }
    public void setID(int id) { this.idUsuario = id; }

    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
}

```

- **ArquivoUsuario:** Sobrescreve as operações de criação, atualização e exclusão para garantir que o arquivo principal permaneça sempre sincronizado em tempo real, gera índices por Email (Hash) e Nome (Árvore B+).

```

public class ArquivoUsuario extends Arquivo<Usuario> {

    HashExtensivel<ParEmailId> indiceEmail;
    ArvoreBMais<ParNomeId> indiceNome;

    public ArquivoUsuario() throws Exception {
        super(nomeEntidade: "usuario", Usuario.class.getConstructor());
        indiceEmail = new HashExtensivel<>(
            ParEmailId.class.getConstructor(),
            n: 4,
            nd: "../dados/usuario/indiceEmail.d.db",
            nc: "../dados/usuario/indiceEmail.c.db");
        indiceNome = new ArvoreBMais<>(
            ParNomeId.class.getConstructor(),
            o: 4,
            na: "../dados/usuario/indiceNome.db");
    }

    @Override
    public int create(Usuario u) throws Exception {
        int id = super.create(u);
        indiceEmail.create(new ParEmailId(u.getEmail(), id));
        indiceNome.create(new ParNomeId(u.getNome(), id));
        return id;
    }

    public Usuario readEmail(String email) throws Exception {
        ParEmailId pei = indiceEmail.read(Math.abs(email.hashCode()));
        if(pei==null)
            return null;
        Usuario u = read(pei.getId());
    }
}

```

- **ControleUsuario:** Lógica de login, cadastro e recuperação de conta.


```

public class ControleUsuario {

    public int login() throws Exception {
        Console.clear();
        System.out.print(s: "Email: ");
        String email = console.nextLine();
        if(email.length()==0)
            return -1;
        System.out.print(s: "Senha: ");
        String senha = console.nextLine();
        Usuario u = arqUsuarios.readEmail(email);
        if(u==null)
            return -1;
        String hash = VisaoUsuario.hash(senha);
        if(hash.equals(u.getHashSenha()))
            return u.getID();
        return -1;
    }

    public void cadastrar() throws Exception {
        Console.clear();
        VisaoUsuario v = new VisaoUsuario();
        Usuario u = v.leUsuarioParaCadastro(console);
        if(u==null)
            return;
        arqUsuarios.create(u);
        System.out.println(x: "Usuario cadastrado!");
    }

    public void recuperarSenha() throws Exception {
        Console.clear();
        System.out.print(s: "Email: ");
    }
}

```

- **VisaoUsuario**: Interface que lê dados e gera hashes para segurança, é possível recuperar senha.

```

public class VisaoUsuario {
    public Usuario leUsuarioParaCadastro(Scanner console) throws Exception {
        System.out.print(s: "Nome: ");
        String nome = console.nextLine();
        if(nome.length()==0)
            return null;

        String email = "";
        boolean valido = false;
        do {
            System.out.print(s: "Email: ");
            email = console.nextLine();
            if(email.length()==0)
                return null;
            if(!email.contains(s: "@")) {
                System.out.println(x: "Email invalido!");
            } else {
                valido = true;
            }
        } while(!valido);

        System.out.print(s: "Senha: ");
        String senha = console.nextLine();
        if(senha.length()==0)
            return null;

        System.out.print(s: "Pergunta secreta (para recuperacao): ");
        String pergunta = console.nextLine();
        if(pergunta.length()==0)
            pergunta = "";
        System.out.print(s: "Resposta secreta: ");
        String resposta = console.nextLine();
    }
}

```

3. Entidade curso:

- **Curso:** Entidade com Nome, Descrição, Data, Código Único e Estado.

```

public class Curso implements InterfaceEntidade {

    private int idCurso;
    private String nome;
    private int dataInicioEpoch;
    private String descricao;
    private String codigoCompartilhavel;
    private byte estado; // 0: aberto, 1: sem novas inscrições, 2: concluído, 3: cancelado
    private int idUsuario; // dono

    public Curso() {
        this(-1, nome: "", (int)java.time.LocalDate.now().toEpochDay(), descricao: "", codigo: "", (byte)0, idUs
    }

    public Curso(String nome, java.time.LocalDate dataInicio, String descricao, int idUsuario) {
        this(-1, nome, (int)dataInicio.toEpochDay(), descricao, gerarCodigoCompartilhavel(), (byte)0, idUs
    }

    public Curso(int id, String nome, int dataInicioEpoch, String descricao, String codigo, byte estado, i
        this.idCurso = id;
        this.nome = nome;
        this.dataInicioEpoch = dataInicioEpoch;
        this.descricao = descricao;
        this.codigoCompartilhavel = codigo;
        this.estado = estado;
        this.idUsuario = idUsuario;
    }

    public int getID() { return idCurso; }
    public void setID(int id) { this.idCurso = id; }

```

- **ArquivoCurso:** Insere o vetor de bytes gerado pelo curso e o salva no arquivo de dados. Logo em seguida, ele obrigatoriamente insere as chaves correspondentes na Tabela Hash Extensível e nas duas Árvores B+, para que as buscas futuras sejam

otimizadas e não precisem varrer o arquivo sequencialmente.

```
public class ArquivoCurso extends Arquivo<Curso> {

    HashExtensivel<ParCodigoId> indiceCodigo;
    ArvoreBMais<ParNomeId> indiceNome;
    ArvoreBMais<ParIdId> indiceUsuarioCurso;

    public ArquivoCurso() throws Exception {
        super(nomeEntidade: "curso", Curso.class.getConstructor());
        indiceCodigo = new HashExtensivel<>({
            ParCodigoId.class.getConstructor(),
            n: 4,
            nd: "../dados/curso/indiceCodigo.d.db",
            nc: "../dados/curso/indiceCodigo.c.db");
        indiceNome = new ArvoreBMais<>({
            ParNomeId.class.getConstructor(),
            o: 4,
            na: "../dados/curso/indiceNome.db");
        indiceUsuarioCurso = new ArvoreBMais<>({
            ParIdId.class.getConstructor(),
            o: 4, na: "../dados/curso/indiceUsuarioCurso.db");
    }

    @Override
    public int create(Curso c) throws Exception {
        // garante código único
        if(c.getCodigoCompartilhavel() == null || c.getCodigoCompartilhavel().length()==0) {
            String codigo;
            do {
                codigo = Curso.gerarCodigoCompartilhavel();
            } while(indiceCodigo.read(Math.abs(codigo.hashCode()))!=null);
        }
    }
}
```

- **ControleCurso:** Gere o menu de cursos, alteração de estados e criação.

```

public class ControleCurso {

    public void menuCursos(int userId) throws Exception {
        VisaoCurso v = new VisaoCurso();
        do {
            Console.clear();
            System.out.println(x: "\nEntrePares 1.0");
            System.out.println(x: "-----");
            System.out.println(x: "\n> Inicio > Meus Cursos\n");

            Curso[] cursos = arqCursos.readUsuario(userId);
            for (int i = 0; i < cursos.length; i++) {
                System.out.println("(" + (i+1) + ") " + cursos[i].getNome() + " - " + cursos[i].getDataInicio());
            }

            System.out.println(x: "\n(A) Novo curso");
            System.out.println(x: "(R) Retornar ao menu anterior");
            System.out.print(s: "\nOpcao: ");
            String op = console.nextLine();
            if(op.length()==0) continue;
            char ch = op.charAt(index: 0);
            if(ch=='A' || ch=='a'){
                Curso novo = v.leCurso(console, userId);
                if(novo!=null) {
                    arqCursos.create(novo);
                    System.out.println("Curso criado! Código: " + novo.getCodigoCompartilhavel());
                }
            }
            else if(ch=='R' || ch=='r') {
                break;
            }
            else {

```

- **VisaoCurso:** Interface para entrada e exibição de dados de cursos.

```

public class VisaoCurso {
    public Curso leCurso(Scanner console, int idUsuario) throws Exception {
        boolean dadosValidos = false;
        do {
            System.out.print(s: "Data de inicio (dd/mm/aaaa): ");
            String aux = console.nextLine();
            if(aux.length()==0) {
                dataInicio = LocalDate.now();
                dadosValidos = true;
            } else {
                try {
                    String[] parts = aux.split(regex: "/");
                    dataInicio = LocalDate.of(
                        Integer.parseInt(parts[2]),
                        Integer.parseInt(parts[1]),
                        Integer.parseInt(parts[0]));
                    dadosValidos = true;
                } catch(Exception e) {
                    System.out.println(x: "Data invalida!");
                }
            }
        } while(!dadosValidos);

        System.out.print(s: "Descricao: ");
        String descricao = console.nextLine();
        if(descricao.length()==0)
            descricao = "";

        Curso c = new Curso(nome, dataInicio, descricao, idUsuario);
        return c;
    }
}

```

Operações Especiais Implementadas

- **Reuso de Espaço (Lista):** Ao excluir ou atualizar um registo (se o tamanho diminuir), o endereço é guardado numa lista de espaços vazios no cabeçalho do arquivo. Novos registos tentam reutilizar estes espaços antes de adicionar no final do arquivo.

```

public class Arquivo<T extends InterfaceEntidade> {

    public void insereEspacoVazio(long endereco, short tamanho) throws Exception {
        arquivo.seek(pos: 4);
        long end = arquivo.readLong();

        // lista vazia
        if(end == -1) {
            arquivo.seek(pos: 4);
            arquivo.writeLong(endereco);
            arquivo.seek(endereco+3);
            arquivo.writeLong(-1);
        }

        // insere na lista existente, de forma ordenada
        else {
            long endAnterior = 4;
            while(end!=-1) {
                arquivo.seek(end+1);
                short tam = arquivo.readShort();
                if(tamanho<tam)
                    break;
                endAnterior = end;
                end = arquivo.readLong();
            }
            arquivo.seek(endereco+3);
            arquivo.writeLong(end);
            if(endAnterior==4)
                arquivo.seek(pos: 4);
            else
                arquivo.seek(endAnterior+3);
            arquivo.writeLong(endereco);
        }
    }
}

```

- **Relacionamento 1:N:** Foi utilizada uma Árvore B+ (indiceUsuarioCurso) que armazena pares de IDs (idUserio e idCurso), permitindo listar rapidamente todos os cursos de um usuário específico.

```

public class ArquivoCurso extends Arquivo<Curso> {

    HashExtensivel<ParCodigoId> indiceCodigo;
    ArvoreBMais<ParNomeId> indiceNome;
    ArvoreBMais<ParIdId> indiceUsuarioCurso;

    public ArquivoCurso() throws Exception {
        super(nomeEntidade: "curso", Curso.class.getConstructor());
        indiceCodigo = new HashExtensivel<>(
            ParCodigoId.class.getConstructor(),
            n: 4,
            nd: "../dados/curso/indiceCodigo.d.db",
            nc: "../dados/curso/indiceCodigo.c.db");
        indiceNome = new ArvoreBMais<>(
            ParNomeId.class.getConstructor(),
            o: 4,
            na: "../dados/curso/indiceNome.db");
        indiceUsuarioCurso = new ArvoreBMais<>(
            ParIdId.class.getConstructor(),
            o: 4, na: "../dados/curso/indiceUsuarioCurso.db");
    }
}

```

- **Segurança com SHA-256:** As senhas e respostas secretas nunca são gravadas em texto limpo. O sistema gera um hash criptográfico antes da persistência.


```
VisaoUsuario.java 1 X
entidades > usuarios > J VisaoUsuario.java > Java > {} usuarios
6 public class VisaoUsuario {
8     public Usuario leUsuarioParaCadastro(Scanner console) throws Exception {
45
46         Usuario u = new Usuario(-1, nome, email, hashSenha, pergunta, hashResposta);
47         return u;
48     }
49
50     public void mostraUsuario(Usuario u) {
51         System.out.println(u);
52     }
53
54     public static String hash(String s) throws Exception {
55         if(s==null)
56             return "";
57         MessageDigest md = MessageDigest.getInstance(algorithm: "SHA-256");
58         byte[] bytes = md.digest(s.getBytes(charsetName: "UTF-8"));
59         return bytesToHex(bytes);
60     }
61
62     private static String bytesToHex(byte[] bytes) {
63         StringBuilder sb = new StringBuilder();
64         for(byte b : bytes)
65             sb.append(String.format(format: "%02x", b));
66         return sb.toString();
67     }
68
69 }
70
```

- **Geração de Código Único:** Cada curso recebe um código alfanumérico de 10 caracteres gerado via SecureRandom. O sistema verifica no índice de códigos se já existe antes de confirmar a criação.

```

J Curso.java 5 X
entidades > cursos > J Curso.java > Language Support for Java(TM) by Red Hat > {} entidades.cursos
11 public class Curso implements InterfaceEntidade {
89     public void fromByteArray(byte[] vb) throws Exception {
93         nome = dis.readUTF();
94         dataInicioEpoch = dis.readInt();
95         descricao = dis.readUTF();
96         codigoCompartilhavel = dis.readUTF();
97         estado = dis.readByte();
98         idUsuario = dis.readInt();
99     }
100
101     public static String gerarCodigoCompartilhavel() {
102         final String alphabet = "abcdefghijklmnopqrstuvwxyzABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
103         SecureRandom rnd = new SecureRandom();
104         StringBuilder sb = new StringBuilder(capacity: 10);
105         for (int i = 0; i < 10; i++) {
106             sb.append(alphabet.charAt(rnd.nextInt(alphabet.length())));
107         }
108         return sb.toString();
109     }
110 }
111
112

```

Checklist:

1. Há um CRUD de usuários que funciona corretamente?
 - Sim.
2. Há um CRUD de cursos que funciona corretamente?
 - Sim.
3. Os cursos estão vinculados aos usuários usando o idUsuario como chave estrangeira?
 - Sim, o idUsuario é armazenado em cada curso e é utilizado para filtragem.
4. Há uma árvore B+ que registre o relacionamento 1:N entre usuários e cursos?
 - Sim.
5. O trabalho compila corretamente?
 - Sim.
6. O trabalho está completo e funcionando sem erros de execução?
 - Sim.

7. O trabalho é original e não a cópia de um trabalho de outro grupo?

Sim.